

The Ultimate 30-Day Blueprint: AI-Powered Security Operations Center

A Comprehensive Guide to Building a Multi-Agent AI System
from Scratch

Prepared for SDE Portfolio Development

Generated: June 2026

Table of Contents

Executive Summary & Architecture Overview

Phase 1: Project Foundation & Log Simulation (Days 1-5)

Phase 2: Core API & Real-Time Engine (Days 6-10)

Phase 3: The Multi-Agent AI System (Days 11-18)

Phase 4: Frontend Next.js Dashboard (Days 19-25)

Phase 5: Advanced Features & Deployment (Days 26-30)

Interview Strategies & Resume Integration

Executive Summary & Architecture Overview

This 15-page document serves as your definitive, day-by-day guide to constructing an enterprise-grade AI-Powered Security Operations Center (SOC). This is not a standard CRUD application. By completing this project, you will demonstrate mastery over distributed systems, real-time data pipelines, and advanced LLM orchestration.

Why This Project Works for SDE Roles

Recruiters see thousands of simple to-do apps and basic e-commerce clones. A multi-agent system utilizing WebSockets, Python log generation, and Next.js immediately signals that you understand how complex systems interact. You are proving that you can manage asynchronous AI tasks while maintaining a highly responsive frontend.

The Architecture

Our system is composed of four distinct layers working in harmony:

- **The Data Generator (Python):** Continuously writes synthetic corporate network logs to a PostgreSQL database.
- **The Backend Engine (Node.js/Express):** Exposes RESTful APIs and maintains active WebSocket connections with the frontend.
- **The AI Orchestrator (LangChain/Python):** A multi-agent framework that polls the database, detects anomalies using OpenAI, conducts database investigations, and writes incident reports.
- **The UI Dashboard (Next.js):** A dark-mode control center that visualizes real-time attacks, generates charts via Recharts, and allows users to query security protocols via a RAG-powered chatbot.

Mental Model: Treat this project like you are founding a startup. Every phase represents a core feature of your product. Don't rush to the AI; the backend and data generation must be solid first.

Phase 1: Project Foundation & Log Simulation

Goal: Establish the environment and create the data stream. AI agents are useless without data. We will build a synthetic log generator that mimics a real corporate network under attack.

Day 1: System Architecture & Repository Setup

Today is about laying the groundwork. A poorly structured project will become unmanageable by Day 15.

- Initialize a monorepo (e.g., using Turborepo or simple folder structure: /frontend, /backend, /ai-agents).
- Initialize Git repository and set up `.gitignore` for Node modules and Python environments.
- Define the exact tech stack: Next.js (Frontend), Node/Express (Backend), Python (AI Agents), PostgreSQL (Database).

```
# Directory Structure
mkdir ai-soc-project
cd ai-soc-project
mkdir frontend backend ai-agents
git init
```

Day 2: Database Schema Design

Design the PostgreSQL database to hold users, raw logs, and AI-generated incidents.

- Install PostgreSQL locally or use a cloud provider like Supabase/Neon.
- Set up Prisma ORM in the backend folder.
- Write the `schema.prisma` file.

```
model Log {
  id          String    @id @default(uuid())
  timestamp   DateTime  @default(now())
  sourceIp    String
  action      String
  username    String?
  status      String
}

model Incident {
  id          String    @id @default(uuid())
  title       String
  severity    String
  description  String
  createdAt   DateTime  @default(now())
}
```

Day 3: Building the Log Simulator (Python) - Part 1

We need raw materials. We will use Python's Faker library to generate fake security logs.

- Set up Python virtual environment (python -m venv venv).
- Install dependencies: pip install faker psycpg2-binary.
- Write a script to generate normal traffic (successful logins, database queries).

```
import random
from faker import Faker
import json

fake = Faker()
def generate_normal_log():
    return {
        "ip": fake.ipv4(),
        "user": fake.user_name(),
        "action": "login",
        "status": "success"
    }
```

Day 4: Building the Log Simulator - Part 2 (Attacks)

Now, inject malicious patterns into the logs. This is what the AI will look for.

- Write functions to simulate Brute Force attacks (multiple failed logins from one IP).
- Write functions to simulate SQL Injection attempts.
- Connect the Python script to PostgreSQL and stream logs at a rate of 5 logs/second.

```
def generate_brute_force():
    target_user = fake.user_name()
```

```
attacker_ip = fake.ipv4()
logs = []
for _ in range(50):
    logs.append({
        "ip": attacker_ip,
        "user": target_user,
        "action": "login",
        "status": "failed"
    })
return logs
```

Day 5: Testing the Data Pipeline & Wrap-up Phase 1

Ensure that your Python script runs continuously and populates your PostgreSQL database without crashing.

- Run the generator script.
- Open a database viewer (like DBeaver or Prisma Studio).
- Verify that thousands of logs are appearing with realistic timestamps.
- Document the log formats in your README.

Milestone Check: You now have a live stream of synthetic security data flowing into a robust database. You are ready to build the API that will serve this data.

Phase 2: Core API & Real-Time Engine

Goal: Build the Express backend that will sit between your database, your AI agents, and your Next.js dashboard. This layer must support WebSockets for real-time alerting.

Day 6: Express.js Setup & REST APIs

Create the foundational endpoints to fetch logs and incidents.

- Initialize Node project: `npm init -y`.
- Install Express, CORS, dotenv: `npm i express cors dotenv`.
- Create standard GET routes: `/api/logs` and `/api/incidents`.

```
const express = require('express');
const app = express();
app.use(express.json());

app.get('/api/logs', async (req, res) => {
  const logs = await prisma.log.findMany({ take: 100 });
  res.json(logs);
});
```

Day 7: Implementing Pagination & Filtering

Dashboards cannot load 100,000 logs at once. Implement server-side pagination.

- Add `limit` and `offset` query parameters to your routes.

- Add filtering by `status` or `ip`.
- Test with Postman or `cURL`.

Day 8: WebSockets Setup (Socket.io)

Standard REST APIs require polling (refreshing the page). WebSockets allow the server to push alerts instantly.

- Install Socket.io: `npm install socket.io`.
- Attach Socket.io to your Express server.
- Create a test event to verify client-server connection.

```
const http = require('http');
const { Server } = require("socket.io");
const server = http.createServer(app);
const io = new Server(server, { cors: { origin: "*" } });

io.on('connection', (socket) => {
  console.log('New dashboard client connected');
});
```

Day 9: Integrating WebSockets with Database Events

When the AI agents write a new 'Incident' to the database, the Node server needs to know so it can emit a socket event.

- Create an endpoint `POST /api/incidents/notify` that the AI Python script will call when it finds an attack.
- Inside this endpoint, trigger `io.emit('new_incident', incidentData)`.

Day 10: Authentication (JWT) & Security

Secure your API so not just anyone can view your fake corporate logs.

- Implement JWT (JSON Web Tokens) for login.
- Create middleware to protect routes.
- Update Postman collections with Auth headers.

Phase 3: The Multi-Agent AI System

Goal: Build the "Brain". Instead of one massive script, you will build specialized AI agents. One detects, one investigates, one acts.

Day 11: Introduction to LangChain & OpenAI API

Set up the LLM environment.

- Get an OpenAI API Key (or use a free local model via Ollama).
- Install Python libraries: `pip install langchain openai`.
- Write a basic script to prompt the LLM to classify a single log line.

```
from langchain_openai import ChatOpenAI
llm = ChatOpenAI(temperature=0)
response = llm.invoke("Is 50 failed logins in 1 minute suspicious?")
print(response.content)
```

Day 12: Building Agent 1 - The Detection Agent

This agent constantly polls the latest logs to look for anomalies.

- Write a Python loop that fetches the last 5 minutes of logs every 60 seconds.
- First, use simple rule-based logic (e.g., IF failed_logins > 20 -> Trigger).
- Pass triggered batches to the LLM to ask: "What type of attack is this?"

Day 13: Structuring AI Outputs

LLMs return plain text, but our database needs structured data (JSON). We must force the LLM to reply in strict JSON format.

- Use LangChain's Output Parsers (PydanticOutputParser).
- Define a Python class for IncidentDetection.
- Ensure the LLM always returns: {"attack_type": "...", "severity": "...", "confidence": 95}.

Day 14: Building Agent 2 - The Investigation Agent

When Agent 1 flags an IP, Agent 2 digs deeper into the database.

- Create an agent equipped with "Tools" (functions it can run).
- Give it a tool: `query_database(ip_address)`.
- Prompt: "You are an investigator. An incident was detected for IP X. Run queries to see if this IP accessed any admin accounts, then summarize your findings."

```
# Prompting the Investigation Agent
prompt = f"""
Incident Detected: Brute Force from {ip}.
Your task: Find out if the attacker eventually logged in successfully.
Use your database query tool to check logs for this IP.
"""
```

Day 15: LangGraph - Orchestrating the Agents

Connect Agent 1 and Agent 2 so they flow automatically.

- Learn the basics of LangGraph (State Graphs).
- Create a workflow: Node 1 (Detect) -> Edge -> Node 2 (Investigate).
- Test the flow end-to-end locally.

Day 16: Building Agent 3 - The Response Agent

Based on the investigation, decide what to do.

- Provide Agent 3 with the Investigation Summary.
- Prompt it to generate a mitigation strategy (e.g., "Block IP 192.x.x.x on Firewall", "Reset password for user admin").
- Format this as a recommended action plan.

Day 17: Building Agent 4 - The Reporting Agent

Executives need readable summaries, not raw data.

- Gather data from Agents 1, 2, and 3.
- Prompt LLM to write a professional, 3-paragraph "Executive Incident Report".
- Save this report text into the Reports table in PostgreSQL.

Day 18: System Integration - AI to API

Connect the AI Python workflow to your Node API.

- When Agent 4 finishes, use the Python requests library to POST the final incident payload to `http://localhost:3000/api/incidents/notify`.
- This completes the loop: Data -> AI -> Node API -> Socket (Ready for UI).

Phase 4: Frontend Next.js Dashboard

Goal: Visualize the chaos. Build a beautiful, dark-mode dashboard that displays real-time attacks and AI reports.

Day 19: Next.js Setup & UI Shell

Create the modern interface using React and Tailwind CSS.

- `npx create-next-app@latest ai-soc-ui.`
- Install Lucide-React for security icons (Shield, AlertTriangle, Activity).
- Build the Layout component: Sidebar navigation, Header, and main content area.

Day 20: Dashboard Home & Metrics Cards

Display high-level statistics.

- Fetch the total number of incidents from the backend API.
- Create components for "Total Alerts", "Critical Threats", and "System Status".
- Use basic CSS/Tailwind animations to make the status "pulse".

Day 21: Real-Time Incident Feed (WebSockets Client)

Make the dashboard come alive. As attacks happen, they should pop up immediately.

- Install `socket.io-client` in Next.js.
- Use a `useEffect` hook to connect to the Node server.
- Listen for `new_incident`. When received, prepend it to your React state array.

```
useEffect(() => {  
  const socket = io("http://localhost:3000");  
  socket.on("new_incident", (data) => {  
    setIncidents(prev => [data, ...prev]);  
    toast.error("New Attack Detected!");  
  });  
  return () => socket.disconnect();  
}, []);
```

Day 22: Data Visualization with Charts

Managers love charts. Add visual analytics.

- Install `Recharts` or `Chart.js`.
- Create a Pie Chart showing "Incident Severity Distribution" (High vs Medium vs Low).
- Create a Bar Chart showing "Attacks over the last 24 hours".

Day 23: Incident Details Page - The AI View

When a user clicks on an alert, show them the AI's full analysis.

- Create dynamic route: `/incidents/[id]`.
- Fetch specific incident details, including the Investigation and Response reports generated by the agents.
- Format the AI response beautifully using Markdown rendering (`react-markdown`).

Day 24: Raw Log Viewer

Engineers still need to see the raw data.

- Build a tabular view of the Logs table.
- Implement front-end pagination or infinite scroll.
- Highlight logs that were involved in an incident (e.g., color failed logins red).

Day 25: Polishing the UI & Error Handling

- Add loading skeletons (loading states) while API calls are running.
- Implement Toast notifications for errors and WebSocket disconnects.
- Ensure dark mode looks professional and cohesive.

Phase 5: Advanced Features & Deployment

Goal: Elevate the project from "good" to "exceptional". Adding RAG and Dockerizing the app will make you stand out in senior-level interviews.

Day 26: Implementing RAG (Retrieval-Augmented Generation)

Give your AI access to actual cybersecurity manuals.

- Set up ChromaDB locally.
- Download the OWASP Top 10 guidelines as PDFs.
- Write a script to chunk these PDFs, generate embeddings (OpenAI), and store them in ChromaDB.

Day 27: The Security Chatbot

Add a chat interface in the Next.js dashboard.

- Build a chat UI component.
- When a user asks "How do I fix a Brute Force attack?", query ChromaDB for relevant OWASP docs.
- Pass the docs + user question to the LLM. Display the answer in the chat.

Day 28: Dockerization

Make your project runnable with a single command.

- Write a `Dockerfile` for the Next.js frontend.
- Write a `Dockerfile` for the Node API.
- Write a `Dockerfile` for the Python AI Agent service.
- Create a `docker-compose.yml` that spins up all three PLUS the PostgreSQL database.

```
version: '3.8'
services:
  db:
    image: postgres:13
  backend:
    build: ./backend
    ports: ["3000:3000"]
  frontend:
    build: ./frontend
    ports: ["3001:3000"]
  ai-agents:
    build: ./ai-agents
```

Day 29: Final Testing & Video Demo

A project is only as good as its presentation.

- Run `docker-compose up --build`. Test the whole system.
- Record a 3-minute video using OBS or Loom.
- Show logs generating -> AI detecting -> Dashboard updating in real-time -> RAG chatbot answering.

Day 30: GitHub Readme & Resume Integration

The final step. Package it for recruiters and interviewers.

- Write an exceptional README.md. Include the System Architecture diagram (draw it in Excalidraw).
- Add instructions on how to run it.
- Add this to your resume using high-impact bullet points:

Resume Bullet Points Example:

- Engineered a real-time AI Security Operations Center (SOC) processing synthetic network traffic, utilizing Next.js, Node.js, and PostgreSQL.
- Orchestrated a multi-agent LLM system (LangChain) to autonomously detect anomalies, investigate logs, and generate executive mitigation reports.
- Implemented WebSocket architecture for instantaneous threat alerting, reducing simulated response visualization time to sub-seconds.
- Integrated Retrieval-Augmented Generation (RAG) using ChromaDB to query OWASP security frameworks directly within the dashboard.

Final Thoughts for Technical Interviews

When discussing this project in an interview, do not just list the tools. Talk about the **problems** you solved. Talk about how you managed the asynchronous nature of the AI agents. Talk about why you chose WebSockets over REST polling for the dashboard. This project provides massive surface area for deep technical discussions.

Good Luck!